



Sujet de TP : Projet "Plateforme-CVTech"

Architecture en Monolithe Modulaire Enterprise-Grade (.NET 10)



Le Contexte Métier

Le problème adressé

Le marché du travail dans la tech est aujourd'hui **fragmenté entre plusieurs plateformes incompatibles** : un candidat doit jongler entre LinkedIn et Indeed pour chercher un poste salarié, entre Malt et Comet pour décrocher une mission freelance, et entre Hacker News, Korben ou ZDNet pour suivre l'actualité de son écosystème technique. De leur côté, les entreprises gèrent en parallèle un ATS pour le recrutement permanent et une marketplace freelance pour les besoins ponctuels, sans vue unifiée des talents disponibles.

Plateforme-CVTech propose de réunir ces trois usages sur **un seul site, une seule identité, un seul CV** :

1.  **Un job board classique (recrutement permanent)** — Les **Entreprises** publient des annonces d'emploi (CDI, CDD, stage, alternance, apprentissage). Les **Candidats** constituent un CV structuré et postulent en un clic, avec lettre de motivation optionnelle. Les annonces sont **publiques** et indexables par les moteurs de recherche : aucun compte n'est requis pour les consulter, seule la candidature exige une inscription.
2.  **Une marketplace d'appels d'offre freelance (mission ponctuelle)** — Les **Entreprises** publient un cahier des charges (contexte, livrables attendus, deadline, fourchette budgétaire). Les **Candidats indépendants** y répondent en soumettant une proposition chiffrée (TJM, durée, méthodologie). L'entreprise compare les propositions reçues et sélectionne le lauréat. Là aussi, les appels d'offre sont **publiquement visibles** ; seule la soumission d'une proposition demande d'être authentifié.
3.  **Un fil d'actualité éditorial (information générale)** — La plateforme publie (ou agrège depuis des sources externes) des **articles d'actualité tech** : tendances du recrutement IT, évolutions salariales, nouveautés des frameworks, retours d'expérience. Ce fil est exposé sous forme de **flux RSS public** pour être consommé par n'importe quel agrégateur (Feedly, NetNewsWire, etc.). **⚠ Important : le RSS ne diffuse PAS les annonces d'emploi ni les appels d'offre — il est strictement dédié au contenu éditorial / informationnel.** Les annonces et appels d'offre, eux, sont déjà publics sur le site et exposés via des endpoints REST standards.
4.  **Un système d'abonnement par domaine métier (push personnalisé)** — En complément, un utilisateur authentifié peut **s'abonner à un ou plusieurs domaines métier** (ex : « Cloud Azure », « Data Science », « Cybersécurité », « DevOps »). Il reçoit alors une **notification** (email et/ou in-app) dès qu'une nouvelle annonce d'emploi ou un nouvel appel d'offre est publié dans ce domaine. C'est l'inverse du RSS : ici l'utilisateur ne consulte pas, il est notifié activement, et uniquement sur ce qui l'intéresse.

Valeur apportée à chaque acteur

Acteur

Ce qu'il en retire

Acteur	Ce qu'il en retire
 Candidat	Une seule identité pour postuler à des CDI et décrocher des missions freelance ; un CV unique réutilisable ; des notifications ciblées par domaine métier qui éliminent le bruit ; un fil RSS pour rester informé sans s'astreindre à visiter le site.
 Entreprise	Un point d'entrée unique pour publier ses besoins permanents et ses besoins ponctuels ; un vivier de candidats déjà qualifiés sur leur domaine ; la visibilité publique de ses annonces sans aucune barrière d'inscription pour les lecteurs.
 Administrateur	Une console de modération pour assurer la qualité éditoriale du fil d'actualité, de la qualité des annonces, et la conformité réglementaire (RGPD, anti-discrimination à l'embauche, lutte contre les fausses offres).

Les trois typologies d'utilisateurs et leurs droits

Trois rôles cohabitent et n'ont **pas les mêmes droits** :

Rôle	Description
 Candidat	Constitue son CV, postule aux annonces d'emploi, soumet des propositions sur les appels d'offre, s'abonne à des domaines métier pour recevoir des notifications.
 Entreprise	Publie ses annonces d'emploi et ses appels d'offre, consulte les candidatures et propositions reçues, sélectionne ses lauréats, peut elle aussi s'abonner à des domaines (veille concurrentielle).
 Administrateur	Hérite de tous les droits : modération des annonces et appels d'offre, publication des articles du fil d'actualité, blocage / réactivation de comptes, gestion du référentiel des domaines métier.

1. L'Architecture Technique Imposée

PROF

L'application est un **Monolithe Modulaire**. Chaque module est une boîte noire étanche. La communication entre modules se fait exclusivement par un bus d'événements interne en mémoire ou par des contrats d'interface publics.

Le Découpage en 5 Couches par Module

Chaque module est structuré en 5 couches logiques :

```
[ RACINE DU MODULE (ex: src/Modules/CatalogueEmploi/) ]
├── 1. Client (Couche d'Exposition)
│   ├── Controllers API (Minimal APIs), DTOs de requêtes/réponses,
│   │   Mappings.
│   └── C'est la SEULE porte d'entrée visible du module depuis
│       l'extérieur.
├── 2. Application (Logique des Cas d'Utilisation - Slices Verticales)
│   └── Dossiers par fonctionnalité (ex: Features/PostulerAnnonce/),
```

Commands, Queries, Handlers, Validators.

- └─ 3. Domaine (Domain-Driven Design Strict - 100% Français)
 - └─ Entités riches, Agrégats, Objets de Valeur (Value Objects), Exceptions métiers.
- └─ 4. Infrastructure (Détails Techniques)
 - └─ Persistance (Entity Framework Core / Dapper sur Azure SQL), Services externes, Génération RSS.
- └─ 5. Loader / Composition Root (Plomberie Technique)
 - └─ Bootstrapper d'injection de dépendances (`ModuleLoader.cs`).

Règle clé sur les permissions

Le module **GestionIdentite** est le **seul** à connaître la matrice des rôles. Les autres modules ne lisent **jamais** directement sa base : ils consomment un contrat public (ex : **IVerificateurPermission**) injecté via le **ModuleLoader**. Chaque handler de cas d'usage doit vérifier la permission **avant** toute action métier.

2. Les 4 Modules Métier à Implémenter

- ◆ Module A : **GestionIdentite** (Profils, Rôles & Permissions)
 - **Métier** : Inscription, gestion des trois typologies d'utilisateurs (Candidat, Entreprise, Administrateur) et de leur matrice de permissions. Expose un contrat public consommé par les 3 autres modules pour autoriser ou refuser une action.
 - **Concepts du domaine (Français)** : **ProfilCandidat**, **ProfilEntreprise**, **Administrateur**, **RoleUtilisateur**, **MatricePermissions**, **RevendicationPermission**.
- ◆ Module B : **CatalogueEmploi** (Annonces & Candidatures)
 - **Métier** : Publication des offres d'emploi par les entreprises, constitution du CV par les candidats, dépôt et suivi de candidatures. Émet l'événement **AnnoncePubliee** pour alimenter le module de diffusion.
 - **Concepts du domaine (Français)** : **AnnonceEmploi**, **CurriculumVitae**, **Candidature**, **TypeContrat**, **DomaineMetier**.
- ◆ Module C : **AppelOffreFreelance** (Missions & Propositions)
 - **Métier** : Une entreprise publie un cahier des charges pour une mission ponctuelle. Les candidats indépendants soumettent une proposition chiffrée (TJM, durée, livrables). L'entreprise compare et sélectionne le lauréat. Émet **AppelOffrePublie**.
 - **Concepts du domaine (Français)** : **AppelOffre**, **CahierDesCharges**, **PropositionFreelance**, **CritereSelection**, **BaremeTJM**.
- ◆ Module D : **ActualiteEtAbonnement** (Fil RSS éditorial & Notifications par domaine)

Ce module porte **deux responsabilités cohérentes** mais bien distinctes — elles partagent la notion de domaine métier mais ne servent pas le même usage utilisateur.

- **Sous-domaine D.1 — Fil d'Actualité éditorial (RSS) :**

Le module gère la publication d'**articles d'actualité tech** (rédigés par les administrateurs ou agrégés depuis des sources externes au choix de l'étudiant). Il expose un **endpoint RSS 2.0 public et anonyme** (`GET /feed/rss`) consommable par n'importe quel agrégateur (Feedly, NetNewsWire, Thunderbird...). Filtrage optionnel par domaine métier : `GET /feed/rss?domaine=cloud-azure`. **Ce fil ne contient PAS les annonces d'emploi ni les appels d'offre** — il est strictement dédié au contenu éditorial.

- **Sous-domaine D.2 — Abonnements & Notifications par domaine métier :**

Un utilisateur authentifié (Candidat ou Entreprise) peut s'abonner à un ou plusieurs **DomaineMetier**. Le module **écoute** les événements **AnnoncePubliee** (émis par **CatalogueEmploi**) et **AppelOffrePublie** (émis par **AppelOffreFreelance**) via le bus interne ; pour chaque abonné concerné par le domaine, il génère une **Notification** (canal e-mail et/ou in-app, au choix de l'utilisateur).

- **Concepts du domaine (Français) :**

- Côté RSS : **ArticleActualite**, **FilActualite**, **SourceExterne** (si agrégation), **CategorieEditoriale**.
- Côté abonnements : **Abonnement**, **Notification**, **CanalDiffusion** (e-mail, in-app), **PreferenceNotification**.
- Partagé : **DomaineMetier** (Value Object utilisé aussi par B et C — exposé via un contrat public).

🔑 Matrice de Permissions (à respecter dans **GestionIdentite**)

Action	Visiteur anonyme	Candidat	Entreprise	Administrateur
Consulter une annonce d'emploi ou un appel d'offre	✓	✓	✓	✓
Consulter le fil RSS d'actualité (<code>/feed/rss</code>)	✓	✓	✓	✓
Constituer / modifier son CV	×	✓	×	✓
Postuler à une annonce d'emploi	×	✓	×	×
Soumettre une proposition sur un appel d'offre	×	✓	×	×
Publier une annonce d'emploi	×	×	✓ (ses annonces)	✓
Publier un appel d'offre	×	×	✓ (ses appels)	✓
Consulter les candidatures / propositions reçues	×	×	✓ (siennes)	✓

Action	Visiteur anonyme	Candidat	Entreprise	Administrateur
S'abonner à un domaine métier (notifications)	×	✓	✓	✓
Publier un article dans le fil d'actualité	×	×	×	✓
Modérer / supprimer une annonce ou un AO	×	×	×	✓
Bloquer / réactiver un compte	×	×	×	✓
Gérer le référentiel des domaines métier	×	×	×	✓

3. Le Travail Attendu sur les AI Skills

Le projet utilise l'IA de l'IDE comme un copilote d'architecture. Cependant, **l'IA ne sait rien du projet au démarrage**. C'est le rôle des étudiants de rédiger les fichiers de règles ([.cursor/rules/](#) ou [.agent/skills/](#)) pour dresser les barrières architecturales.

Les étudiants devront compléter et enrichir les squelettes suivants :

 Squelette 1 : Le Gardien de l'Architecture ([architecture-monolithes.md](#))

```
---
name: cvtech-architecture-guard
description: Force le respect de la structure des couches du projet
Plateforme-CVTech
globs: "src/Modules/**/*.cs"
---
# CONTEXTE
Nous sommes en 2026. Le projet utilise .NET 10. Le code doit refléter le
langage métier en français pour les couches Domain et Application, tout
en gardant les standards industriels (anglais) pour l'infrastructure et
la technique (Controllers, Handlers, Infrastructure, ModuleLoader).

# INSTRUCTIONS
[À COMPLÉTER PAR L'ÉTUDIANT : Expliquer ici à l'IA les règles
d'interdiction (ex : pas de DbContext dans le Domain, isolation des
contrôleurs dans la couche Client, structure d'une Vertical Slice avec
MediatR, communication inter-modules uniquement via contrats publics ou
bus d'événements en mémoire, jamais de référence directe entre dossiers
de modules différents...)]
```

 Squelette 2 : Le Gardien des Permissions ([regles-permissions.md](#))

```

---
name: cvtech-permission-guard
description: Vérifie que chaque cas d'usage contrôle les permissions
avant l'action métier
globs: "src/Modules/**/Application/Features/**/*.*cs"
---
# CONTEXTE
Trois rôles cohabitent : Candidat, Entreprise, Administrateur. Aucune
action métier ne doit s'exécuter sans avoir interrogé en amont le
contrat `IVerificateurPermission` exposé par le module
`GestionIdentite`.

# INSTRUCTIONS
[À COMPLÉTER PAR L'ÉTUDIANT : Imposer qu'un Handler refuse de
compiler/passer la revue si la première ligne n'est pas une vérification
de permission. Décrire comment l'IA doit générer une
`PermissionRefuseeException` métier en cas d'échec, et comment elle doit
traduire la matrice de permissions du README en code.]

```

Squelette 3 : Le Pilote du TDD ([regles-tdd.md](#))

```

---
name: cvtech-tdd-expert
description: Force l'écriture des tests de spécification avant le code
métier
globs: "tests/**/*.*cs"
---
# PROTOCOLE TDD
[À COMPLÉTER PAR L'ÉTUDIANT : Définir comment l'IA doit structurer les
tests avec xUnit et FluentAssertions, imposer le nommage des méthodes de
test décrivant une règle métier en français (ex :
`UnCandidatBloquéNePeutPasPostuler`), et interdire la génération du code
de production avant la validation du test (Red → Green → Refactor).]

```

PROF

4. Modalités de Rendu & Livrables Attendus

 **Date Limite de Soumission :** L'intégralité du projet doit être soumise sous la forme d'un dépôt Git unique **en fin de semaine**.

Contenu Obligatoire du Dépôt :

1. **Code Applicatif Complet du Back-end :** L'intégralité du code C# des 4 modules, incluant la base de données configurée et les scripts de migration Azure SQL / EF Core.
2. **Code Applicatif Complet du Front-end :** L'interface graphique fonctionnelle (React ou Blazor WebAssembly) interconnectée avec l'API, avec **trois parcours utilisateur** distincts (Candidat, Entreprise, Administrateur).

3. **Endpoint RSS public d'actualité** : Une URL `/feed/rss` (anonyme, sans authentification) qui retourne un flux **RSS 2.0 valide** contenant exclusivement les **articles éditoriaux du fil d'actualité** publiés par les administrateurs. Filtrage optionnel par domaine : `/feed/rss?domaine=cloud-azure`. ⚠ Ce flux **ne contient ni annonces d'emploi ni appels d'offre** — ceux-ci sont exposés via les endpoints REST publics de leurs modules respectifs.
4. **Système de notifications par abonnement** : Pour au moins **un canal** (e-mail console/SMTP de test, ou WebSocket/in-app), démonstration qu'un abonnement à un domaine métier déclenche bien une notification à la publication d'une annonce ou d'un appel d'offre dans ce domaine.
5. **Dossier des AI Skills** : Vos fichiers de règles complétés (`.cursor/rules/` ou `.agent/skills/`).
6. **Fichier README.md avec Diagramme Mermaid** : Un guide pas-à-pas strict et visuel détaillant l'ordre d'exécution des scripts et du démarrage.

Les étudiants doivent s'inspirer du modèle logique suivant dans leur documentation pour cartographier le lancement :

```
graph TD
    A[1. Clonage du dépôt Git] --> B[2. Configuration des variables d'environnement]
    B --> C[3. Exécution du script d'initialisation de la Base de Données SQL]
    C --> D[4. Restauration & Compilation: dotnet restore]
    D --> E[5. Lancement des Tests Unitaires: dotnet test]
    E -->|Si tous les tests sont au Vert| F[6. Démarrage du Back-end API]
    F --> G[7. Démarrage du Front-end]
    G --> H[8. Validation des 3 parcours: Candidat, Entreprise, Admin]
    H --> I[9. Vérification du flux RSS d'actualité éditoriale]
    I --> J[10. Test d'un abonnement par domaine: notification reçue à la publication]

    style E fill:#4CAF50,stroke:#388E3C,color:#fff
    style F fill:#2196F3,stroke:#1976D2,color:#fff
    style G fill:#2196F3,stroke:#1976D2,color:#fff
    style I fill:#FF9800,stroke:#F57C00,color:#fff
    style J fill:#FF9800,stroke:#F57C00,color:#fff
```

PROF

III. Critères d'Évaluation principaux :

- **Qualité de la DX (Developer Experience)** : Le projet se lance-t-il immédiatement en suivant leur graphique Mermaid ?
- **Langage Métier Cohérent** : Respect de l'Ubiquitous Language (français pour le Domaine/Application, anglais pour la plomberie technique).
- **Étanchéité des Modules** : Aucun accès direct à la base d'un autre module ; communication uniquement par contrats publics ou bus d'événements interne.
- **Respect de la Matrice de Permissions** : Chaque cas d'usage interroge `GestionIdentite` avant toute action métier ; un test prouve qu'une action interdite est refusée.

- **Fonctionnement du RSS éditorial :** Le flux [/feed/rss](#) passe le validateur W3C et ne contient **que** les articles éditoriaux (jamais d'annonces ni d'AO).
- **Fonctionnement des Abonnements / Notifications :** Un abonnement à un domaine déclenche bien une notification (par e-mail ou in-app) à la publication d'une annonce / AO dans ce domaine, et **uniquement** pour les abonnés concernés.
- **Autonomie des Skills :** L'enseignant testera l'efficacité des fichiers de règles en demandant une nouvelle fonctionnalité à l'IA (ex : « ajoute la fonctionnalité de favoris sur une annonce »).
- **Couverture TDD :** Qualité et pertinence des tests unitaires exécutés à vide avant le code de production.